

A CAPSTONE PROJECT REPORT ON

MAHACONNECT: GOVERNMENT PLATFORM INTEROPERABILITY SYSTEM

A Unified Multi-Departmental e-Governance Integration Architecture

Submitted in partial fulfillment of the requirements for the award of the Degree of

BACHELOR OF SCIENCE IN INFORMATION TECHNOLOGY (BSc IT)

(Full Stack Development & Management – FSDM)

SUBMITTED BY:

PAWAN MISHRA

Roll No: 202402104

SAMARTH NIVADUNGE

Roll No: 202402111

Class: TY BSc IT • Semester V

DEPARTMENT OF INFORMATION TECHNOLOGY

ZSCT'S THAKUR SHYAMNARAYAN DEGREE COLLEGE

(Affiliated to University of Mumbai)

MUMBAI – MAHARASHTRA – 400101

Academic Year: 2026 – 2027

Production Deployment URL: <https://mahaconnect-rose.vercel.app>

ZSCT'S THAKUR SHYAMNARAYAN DEGREE COLLEGE

(Affiliated to University of Mumbai)

MUMBAI – MAHARASHTRA – 400101

DEPARTMENT OF INFORMATION TECHNOLOGY**CERTIFICATE**

This is to certify that the project entitled "**MahaConnect: Government Platform Interoperability System**" is a bonafide work carried out by **Pawan Mishra** (Roll No: **202402104**) and **Samarth Nivadunge** (Roll No: **202402111**) in partial fulfillment of the requirements for the degree of **Bachelor of Science in Information Technology (TY BSc IT)**, during the academic year **2026–2027**.

The candidates have completed the project work under the guidance of the Department of Information Technology and have demonstrated their understanding of full-stack web development, MERN technology integration, RESTful API communication, database management, authentication, authorization, and government-platform interoperability modeling.

PROJECT GUIDE

[PROJECT GUIDE NAME]

HEAD OF DEPARTMENT

Dept. of Information Technology

PRINCIPALThakur Shyamnarayan Degree
College

DECLARATION

We, **Pawan Mishra** and **Samarth Nivadunge**, hereby declare that the capstone project report entitled "**MahaConnect – Government Platform Interoperability System**", submitted to the Department of Information Technology in partial fulfillment of the requirements for the award of the Degree of **Bachelor of Science in Information Technology (TY BSc IT)**, is an authentic record of original work carried out by us under institutional academic guidance.

We further declare that this report has not been submitted either concurrently or previously to any other university, college, or examination board for the award of any degree or diploma. All external libraries, architectural design patterns, research papers, and technical standards cited throughout this document have been duly acknowledged in the references section.

PAWAN MISHRA

Roll No: 202402104

SAMARTH NIVADUNGE

Roll No: 202402111

TY BSc IT • Semester V

Date: October 2026

ACKNOWLEDGEMENT

We express our sincere gratitude to the University of Mumbai and ZSCT's Thakur Shyamnarayan Degree College, Department of Information Technology, for providing us with the opportunity to undertake this capstone project.

We are deeply thankful to our Principal, Head of the Department, faculty members, and project guide for their continuous guidance, encouragement, valuable suggestions, and constructive feedback throughout the development of MahaConnect.

We would like to express our special appreciation to our project guide for providing technical guidance in MERN stack development, RESTful API integration, database management, authentication, authorization, and interoperability concepts.

We are also grateful to our classmates, friends, family members, and everyone who directly or indirectly supported us during the planning, development, testing, documentation, and completion of this project.

Finally, we express our sincere thanks to everyone who contributed to the successful completion of our capstone project, "MahaConnect: Government Platform Interoperability System."

ABSTRACT

In contemporary public administration, citizens frequently encounter severe friction when navigating public services due to deep architectural silos across government departments. Traditional e-Governance infrastructures operate as isolated, monolithic islands where citizens are subjected to redundant demographic data submissions, disparate authentication credentials, uncoordinated document verifications, and fragmented tracking mechanisms. When an individual requires interconnected public services—such as a student scholarship requiring income verification from Revenue, caste verification from Social Welfare, and academic verification from Higher Education—the absence of inter-departmental interoperability causes significant administrative delay.

To address these systemic inefficiencies, this capstone project designs and implements **MahaConnect**—a full-stack Government Platform Interoperability System developed using the **MERN stack** (MongoDB, Express.js, React, Node.js) paired with Vite, Tailwind CSS, JSON Web Token (JWT) role-based authorization, and a simulated Government Interoperability Service Layer.

MahaConnect demonstrates how an asynchronous, single-window citizen portal can seamlessly bridge multiple autonomous administrative departments: Revenue, Transport, Education, Municipal Corporation, Employment, Social Welfare, and Urban Development. The platform introduces:

- **Universal Single-Window Citizen Portal:** Eliminates duplicate entry through unified master profile schemas and dynamic multi-step application wizards.
- **Departmental Interoperability Service Layer:** Dispatches citizen filings to simulated departmental REST APIs while recording granular audit trails (HTTP methods, response latencies, source-destination routing, and payload status).
- **Department Officer Scrutiny Console:** Provides department-scoped workflows, inline document verification dossiers, and real-time status transitions with processing remarks.
- **State Administrator Monitoring Cockpit:** Offers centralized system oversight, dynamic service schema configuration, and live API telemetry analytics.
- **Secure RESTful API Communication:** Enforces JSON Web Token bearer authentication, bcrypt password hashing with salt, Helmet security headers, rate limiting, and structured error-handling pipelines.

The system is deployed on live cloud infrastructure (Vercel) backed by a MongoDB document store, with comprehensive REST API contracts validated through automated Postman test suites and browser journey testing.

TABLE OF CONTENTS

Sr. No.	Content / Chapter Title	Page No.
1	Title Page / Cover Page	i
2	Certificate of Approval	ii
3	Declaration & Acknowledgement	iii
4	Abstract	iv
5	List of Figures	vi
6	List of Tables	vii
1.0	CHAPTER 1 – INTRODUCTION	1
1.1	Project Overview	1
1.2	Problem Statement	2
1.3	Project Objectives	2
1.4	Scope and Applicability	3
1.5	Significance of the Project	3
2.0	CHAPTER 2 – LITERATURE REVIEW & EXISTING SYSTEMS	4
2.1	Traditional Siloed Governance Models	4
2.2	Existing e-Governance Platforms	5
2.3	Interoperability in Government Systems	5
2.4	Limitations of Existing Approaches	6
2.5	Proposed MahaConnect Approach	6
3.0	CHAPTER 3 – SYSTEM ARCHITECTURE & INTEROPERABILITY MODEL	7
3.1	High-Level Architectural Paradigm	7
3.2	Gateway Interoperability Sequence Flow	8
3.3	MERN Architecture & Layer Separation	9
3.4	Departmental Interoperability Model	10
3.5	Application Status Lifecycle	10
4.0	CHAPTER 4 – SYSTEM REQUIREMENTS & TECHNOLOGIES USED	11
4.1	Functional Requirements	11

4.2	Non-Functional Requirements	12
4.3	Hardware Requirements	12
4.4	Software Requirements	13
4.5	Technology Stack Details	13
5.0	CHAPTER 5 – SYSTEM ANALYSIS & FEASIBILITY STUDY	14
5.1	Technical Feasibility	14
5.2	Operational Feasibility	15
5.3	Economic Feasibility	15
5.4	Security Feasibility	16
6.0	CHAPTER 6 – SYSTEM DESIGN, DFD & DATABASE MODELING	17
6.1	Data Flow Diagrams (DFD) Overview	17
6.2	DFD Level 0 – Context Diagram	17
6.3	DFD Level 1 – Functional Decomposition	18
6.4	Entity Relationship / Database Model	19
6.5	Database Collections Schema	20
6.6	Data Dictionary	21
7.0	CHAPTER 7 – FULL-STACK IMPLEMENTATION DETAILS	22
7.1	Frontend Architecture & Component Tree	22
7.2	Backend Architecture & Routing Pipeline	23
7.3	REST API Implementation	24
7.4	CRUD Operations & Dynamic Database Records	25
7.5	Form Validation & Error Trapping	26
7.6	Complete Application Workflow	27
8.0	CHAPTER 8 – SECURITY, CRYPTOGRAPHY & ACCESS CONTROL	28
8.1	Password Hashing with bcrypt	28
8.2	JWT Authentication & Bearer Token Verification	29
8.3	Role-Based Access Control (RBAC)	29

8.4	Security Middleware: Helmet, CORS & Rate Limiting	30
8.5	Audit Logging & Telemetry Recording	31
9.0	CHAPTER 9 – TESTING & VALIDATION	32
9.1	Testing Strategy & Methodology	32
9.2	Functional & Frontend Validation Testing	33
9.3	API Testing using Postman Suite	33
9.4	Test Cases and Results Table	34
9.5	Production Deployment Verification	35
10.0	CHAPTER 10 – RESULTS, CONCLUSION & FUTURE SCOPE	36
10.1	Results	36
10.2	Conclusion	37
10.3	Learning Outcomes	37
10.4	Limitations	38
10.5	Future Scope	38
11.0	PROJECT SCREENSHOTS & IMPLEMENTATION PHOTOS	39
12.0	PROJECT DETAILS PAGE	55
13.0	REFERENCES / BIBLIOGRAPHY	56

LIST OF FIGURES

Figure No.	Figure Title	Page No.
Figure 1	MahaConnect System Architecture Diagram	7
Figure 2	Gateway Interoperability Sequence Flow	8
Figure 3	Data Flow Diagram (DFD Level 0 – Context Diagram)	18
Figure 4	Data Flow Diagram (DFD Level 1 – Functional Decomposition)	19
Figure 5	Database Entity-Relationship (ER) Model	20
Figure 6	MahaConnect Public Landing Page & Citizen Gateway	39
Figure 7	Unified Authentication Portal with Seeded Demo Credentials	40
Figure 8	Citizen Self-Service Dashboard with Metric Cards & Quick Navigation	41
Figure 9	Cross-Departmental Government Services Directory with Filtering	42
Figure 10	Dynamic Application Form Wizard: Step 1 (Personal Demographics)	43
Figure 11	Client-Side Form Validation Alert & Error Prevention	44
Figure 12	Document Upload & Multi-Format Verification	45
Figure 13	Application Submission Confirmation & Unique Reference Generation	46
Figure 14	Live Citizen Tracking Timeline with Multi-Stage Progression	47
Figure 15	Department Officer Scrutiny Console & Pending Application Queue	48
Figure 16	Officer Application Scrutiny Dossier with Inline Document Viewer	49
Figure 17	Officer Status Transition Pipeline & Remarks Endorsement	50
Figure 18	State Administrator Governance Dashboard & Interoperability KPIs	51
Figure 19	State Department Management Console & Endpoint Configuration	52
Figure 20	Service Schema Configuration & Dynamic Field Definition	52
Figure 21	REST API Interoperability Gateway Logs & Modal Payload Inspector	53
Figure 22	Postman Test Suite: GET /api/departments (Status 200 OK)	53
Figure 23	Postman Test Suite: POST /api/auth/login (JWT Token Generation)	54
Figure 24	Postman Test Suite: PATCH /api/applications/:id/status	54
Figure 25	Postman Test Suite: DELETE /api/departments/:id (Integrity Check)	54
Figure 26	MongoDB Shell (mongosh) Collection & Query Ledger Inspection	55

Figure 27	MahaConnect Final Working Multi-Portal Application Showcase	55
-----------	---	----

LIST OF TABLES

Table No.	Table Title	Page No.
Table 1	Comparative Analysis of Existing Systems vs. MahaConnect	6
Table 2	User Roles and Permission Scopes	9
Table 3	Simulated Government Departments in MahaConnect	10
Table 4	Application Status Lifecycle States	10
Table 5	Hardware Requirements Specification	12
Table 6	Software Requirements Specification	13
Table 7	Database Collections & Schema Constraints	20
Table 8	Core REST API Endpoints Specification	24
Table 9	Role-Based Access Control (RBAC) Permission Matrix	30
Table 10	Automated Postman Test Case Execution Matrix	34

CHAPTER 1 – INTRODUCTION

1.1 Project Overview

In modern state administration, electronic governance (e-Governance) represents the primary conduit through which government entities interact with the public. Historically, state departments developed digital systems in isolation to automate their internal departmental tasks. Consequently, individual entities—such as the Transport Department, the Revenue Department, the Department of School and Higher Education, Municipal Corporations, and the Department of Social Welfare—operate separate platforms with distinct databases, differing authentication protocols, and non-communicating network interfaces.

MahaConnect is conceptualized and engineered to address this fragmentation. Designed as a full-stack e-Governance platform built on the modern **MERN stack** (MongoDB, Express.js, React, Node.js), MahaConnect serves as a unified digital bridge connecting citizens and government departments. The platform facilitates single-sign-on (SSO) authenticated access, cross-departmental service browsing, dynamic multi-step application filing, document archiving, transparent real-time tracking, and automated inter-departmental API routing.

1.2 Problem Statement

The primary challenge addressed by MahaConnect is the presence of functional information silos in public administration. When a citizen requires a set of related government services (for example, applying for a college scholarship which requires an Income Certificate from Revenue, a Caste Certificate from Social Welfare, and an Enrollment Verification from Higher Education), the citizen is compelled to:

- Create and maintain separate login credentials across multiple departmental websites.
- Repeatedly enter identical demographic data (Full Name, Date of Birth, Aadhaar number, Residential Address, Mobile Number).
- Physically upload identical identity documents to separate departmental servers, multiplying storage overhead and bandwidth consumption.
- Manually visit disjointed tracking portals with separate application numbers, leaving citizens without unified visibility over pending state matters.

1.3 Project Objectives

The primary engineering and operational objectives of MahaConnect include:

1. **Single-Window Citizen Portal:** Develop a unified, responsive React-based interface allowing citizens to access state services through a single authenticated session.
2. **Microservices & Gateway Interoperability:** Implement an API Gateway architecture using Node.js and Express.js middleware that acts as an intelligent intermediary, translating unified citizen requests into department-specific schemas.
3. **Dynamic Multi-Step Form Engine:** Construct a schema-driven form wizard that dynamically renders inputs based on the target service's JSON parameter specification.

4. **Cross-Departmental Document Abstraction:** Provide a standardized document upload and verification pipeline supporting multi-format attachments with validation.
5. **Transparent Real-Time Status Tracking:** Render interactive visual milestone timelines depicting every lifecycle state from initial submission to final officer disposition.
6. **Role-Based Access Control (RBAC):** Enforce cryptographically secured authorization separating Citizens, Department Officers, and State Administrators.
7. **Audit Telemetry & Gateway Ledger:** Maintain a MongoDB audit log recording every API transaction, source-destination routing path, HTTP status code, and latency metrics.

1.4 Scope and Applicability

The scope of this project encompasses the lifecycle of simulated government service delivery within a state ecosystem. It models seven core government departments: Revenue, Transport, Education, Municipal Corporation, Employment, Social Welfare, and Urban Development. As an academic capstone implementation for TY BSc IT Semester V, downstream departmental APIs are simulated through a dedicated service layer, while the RESTful contracts, payload schemas, security tokens, and routing controllers are designed to reflect real-world architectural principles.

1.5 Significance of the Project

The MahaConnect project demonstrates how modern web technologies can mitigate procedural overhead in public administration. By decoupling the citizen interface from departmental backend peculiarities through an API Gateway, the system proves that digital interoperability can be attained without necessitating disruptive replacements of existing departmental infrastructure.

CHAPTER 2 – LITERATURE REVIEW & EXISTING SYSTEMS

2.1 Traditional Siloed Governance Models

Early e-Governance initiatives focused on digitizing individual departmental workflows in isolation. While this computerization replaced paper registers, it created digital silos. Research into public sector software architectures indicates that siloed systems suffer from data redundancy, inconsistent state records, repetitive administrative workloads, and difficulty in cross-verifying applicant eligibility across department boundaries.

2.2 Existing e-Governance Platforms

Various state and national portals currently provide digital citizen services. In India, platforms such as DigiLocker, UMANG, and state-level portals (e.g., Aaple Sarkar) have made substantial progress toward service aggregation. However, many existing state portals still rely on redirected web links to separate legacy departmental engines, meaning the user experience remains non-uniform and data entry is still frequently duplicated across forms.

2.3 Interoperability in Government Systems

Interoperability in distributed software systems refers to the capability of heterogeneous systems to exchange data according to standard formats and protocols. The Ministry of Electronics and Information Technology (MeitY) published the *e-Governance Interoperability Framework for India (e-GIF)*, emphasizing open standards, JSON/REST data exchange, metadata directories, and service-oriented architectures. MahaConnect adopts these foundational principles by establishing an API Gateway that abstracts departmental protocols behind unified REST interfaces.

2.4 Limitations of Existing Approaches

A critical analysis of current implementations reveals the following limitations:

- **Non-standardized Payloads:** Each departmental portal requires custom parameter formats, preventing automated validation.
- **Fragmented Status Tracking:** Applicants receive distinct tracking identifiers per department with no centralized timeline.
- **Lack of Real-Time API Telemetry:** System administrators have limited visibility into cross-departmental communication bottlenecks or error rates.

2.5 Proposed MahaConnect Approach

MahaConnect introduces an intelligent API Gateway layer that unifies citizen identity, dynamically compiles application schemas, coordinates simulated departmental microservices, and maintains an immutable audit ledger of all inter-departmental transactions.

Table 1: Comparative Analysis of Existing Systems vs. MahaConnect

Dimension	Legacy Department Portals	Centralized Monoliths	MahaConnect System
Architecture	Isolated Siloed Servers	Monolithic Relational DB	Modern MERN Gateway Model
User Experience	Multiple Logins & Portals	Rigid Single Interface	Responsive React SPA + Tailwind CSS
Interoperability	None (Paper Bridge)	Database Stored Procedures	RESTful Interoperability Gateway
Schema Flexibility	Hardcoded Database Tables	Static Database Schemas	Dynamic JSON-Schema Service Builder
Audit & Telemetry	Fragmented Text Logs	Standard Database Triggers	Real-Time ApiLog Audit Ledger
Deployment Model	Dedicated Local Servers	Central Government Data Center	Cloud Serverless (Vercel) + MongoDB

CHAPTER 3 – SYSTEM ARCHITECTURE & INTEROPERABILITY MODEL

3.1 High-Level Architectural Paradigm

MahaConnect utilizes a multi-tiered, decoupled client-server architecture designed around microservices and API Gateway integration principles. The architectural flow is:



Figure 1: MahaConnect System Architecture Diagram

Multi-tier architectural diagram depicting the Presentation Tier, Express API Gateway, Simulated Interoperability Service Layer, and MongoDB Document Store.

3.2 Gateway Interoperability Sequence Flow

When a citizen submits an application, the request transitions through a standardized eight-step gateway sequence:



Figure 2: Gateway Interoperability Sequence Flow

Eight-step execution flow from initial citizen submission to payload validation, departmental dispatch, MongoDB persistence, audit telemetry logging, and response delivery.

3.3 MERN Architecture & Layer Separation

The MERN technology stack provides a cohesive JavaScript/JSON runtime across both client and server tiers:

- **React.js (Frontend):** Handles stateful UI components, form step wizards, route transitions, and asynchronous API calls via Axios.
- **Express.js & Node.js (Gateway & Backend):** Implements REST API routes, JWT security verification, input sanitization, and departmental routing logic.
- **MongoDB & Mongoose (Persistence):** Stores semi-structured citizen records, dynamic form parameters, documents, and telemetry logs in BSON format.

Table 2: User Roles and Permission Scopes

User Role	Primary Capabilities	Access Scope
Citizen	Profile management, service browsing, application filing, document uploads, real-time status tracking	Personal profile and owned application records
Department Officer	Application scrutiny, document verification, approval/rejection, formal remarks logging	Applications scoped to assigned department
State Administrator	Department provisioning, service catalog configuration, user management, telemetry monitoring	System-wide administrative authority

3.4 Departmental Interoperability Model

To demonstrate real-world interoperability, MahaConnect models seven core state administrative departments:

Table 3: Simulated Government Departments in MahaConnect

Department Name	Code	Representative Services	Simulated Gateway Endpoint
Revenue Department	REV	Income Certificate, Caste Certificate	/api/v1/revenue/inbound
Transport Department (RTO)	TRP	New Driving Licence, Licence Renewal	/api/v1/transport/inbound
Education Department	EDU	Higher Education Scholarship, Bonafide	/api/v1/education/inbound
Municipal Corporation	MUN	Birth Certificate, Property Tax Assessment	/api/v1/municipal/inbound
Employment Department	EMP	Employment Registration, Skill Portal	/api/v1/employment/inbound
Social Welfare	SOC	Disability Pension, Senior Citizen Card	/api/v1/social/inbound
Urban Development	URB	Zoning Verification, Water Connection	/api/v1/urban/inbound

3.5 Application Status Lifecycle

Table 4: Application Status Lifecycle States

Status State	Trigger Event	Responsible Actor	Citizen Notification
Draft	Citizen starts application form	Citizen	None (Local draft)
Submitted	Citizen confirms and submits	Gateway Dispatcher	Submission Confirmation
Under Review	Officer opens application dossier	Department Officer	Review in progress
Info Required	Officer requests document clarification	Department Officer	Action required alert
Approved	Officer validates proofs and signs off	Department Officer	Approval certificate alert
Rejected	Officer detects discrepancy with remarks	Department Officer	Rejection reason alert
Completed	Final service delivery executed	Gateway / Officer	Completion confirmation

CHAPTER 4 – SYSTEM REQUIREMENTS & TECHNOLOGIES USED

4.1 Functional Requirements

1. **User Authentication:** Secure registration and login using JWT tokens and bcrypt password hashing.
2. **Role-Based Routing:** Restrict portal routes based on verified user roles (Citizen, Officer, Admin).
3. **Service Browsing & Filtering:** Categorized directory of government services with search and department filtering.
4. **Dynamic Multi-Step Form Submission:** Step-by-step form capturing personal demographics, service-specific fields, and file uploads.
5. **Real-Time Application Tracking:** Visual milestone tracker illustrating chronological status updates and officer remarks.
6. **Department Scrutiny Workflow:** Interface for officers to inspect applicant records, view uploaded files, and update status.
7. **Centralized Administration:** Manage departments, configure services, and inspect API telemetry logs.

4.2 Non-Functional Requirements

1. **Usability:** Responsive design supporting desktop, tablet, and mobile viewports via Tailwind CSS.
2. **Performance:** Sub-100ms API response latency for cached lookups and sub-200ms for application writes.
3. **Security:** Stateless JWT verification, input sanitization, HTTP security headers (Helmet), and CORS enforcement.
4. **Reliability:** Centralized error-handling middleware preventing server crashes during malformed requests.
5. **Maintainability:** Modular codebase separating controllers, routes, middleware, and database models.

4.3 Hardware Requirements

Table 5: Hardware Requirements Specification

Component	Minimum Development Specification	Recommended Specification
Processor	Intel Core i3 / AMD Ryzen 3 @ 2.0 GHz	Intel Core i5 / Apple Silicon M-series
Memory (RAM)	4 GB DDR4	8 GB – 16 GB DDR4/Unified
Storage	20 GB available SSD space	50 GB+ NVMe SSD space
Network	Active Internet Connection (2 Mbps)	Broadband Connection (10 Mbps+)
Display	1366 × 768 resolution	1920 × 1080 Full HD resolution

4.4 Software Requirements

Table 6: Software Requirements Specification

Software / Tool	Version / Environment	Purpose
Operating System	macOS Sequoia / Ubuntu 22.04 / Windows 11	Host Development Operating System
Runtime Environment	Node.js v20+ LTS / v24.6	JavaScript Server Execution Runtime
Database Engine	MongoDB Community v7.0 / Atlas	NoSQL Document Database Engine
Package Manager	npm v10+	Dependency Management Tool
API Testing Tool	Postman Desktop v11	REST API Functional Contract Verification
Browser Engine	Google Chrome / Chromium	Client Rendering & Puppeteer Automation
Version Control	Git v2.40+ & GitHub	Source Code Management
Hosting Platform	Vercel Serverless Edge Platform	Cloud Production Deployment

4.5 Technology Stack Details

- **Frontend:** React 18, Vite 6, Tailwind CSS 3.4, React Router v6, Axios, Lucide React icons.
- **Backend:** Node.js, Express.js 4.21, Helmet, Express-Rate-Limit, CORS, Morgan logging.
- **Database:** MongoDB Community, Mongoose ODM 8.9 with strict schemas.
- **Authentication:** jsonwebtoken (JWT 9.0) with HMAC-SHA256 signing, bcryptjs 3.0.

CHAPTER 5 – SYSTEM ANALYSIS & FEASIBILITY STUDY

5.1 Technical Feasibility

The technical feasibility of MahaConnect is established through the utilization of proven web technologies within the MERN ecosystem. Node.js provides non-blocking, asynchronous I/O capabilities well-suited for an API Gateway coordinating multiple concurrent requests. MongoDB's JSON-native BSON document format naturally accommodates the varied schemas required by diverse government services without requiring schema redesigns. React's component state model enables responsive multi-step form navigation and dynamic validation.

5.2 Operational Feasibility

The platform reduces operational friction for citizens and officers. Citizens access services through a clean, intuitive interface without requiring specialized training. Department officers work within dedicated scrutiny queues scoped to their department, streamlining review workflows. Administrators gain system visibility through telemetry log tables and operational charts.

5.3 Economic Feasibility

Utilizing open-source technologies (React, Node.js, Express, MongoDB Community) eliminates expensive proprietary software license fees. Deploying through modern serverless platforms like Vercel aligns resource utilization directly with application traffic, keeping hosting and maintenance costs modest for academic and organizational use cases.

5.4 Security Feasibility

Security is maintained through stateless JSON Web Tokens (JWT) signed with HMAC-SHA256, salted bcrypt password hashing (10 rounds), HTTP header hardening via Helmet, Cross-Origin Resource Sharing (CORS) whitelisting, rate limiting against brute-force attempts, and input sanitization before database insertion.

CHAPTER 6 – SYSTEM DESIGN, DFD & DATABASE MODELING

6.1 Data Flow Diagrams (DFD) Overview

Data Flow Diagrams visually represent the boundaries, external entities, functional processes, and internal data stores of the MahaConnect system.

6.2 DFD Level 0 – Context Diagram

The Context Diagram represents MahaConnect as a single central system interacting with four external entities: Citizens, Department Officers, State Administrators, and Simulated Department REST Microservices.

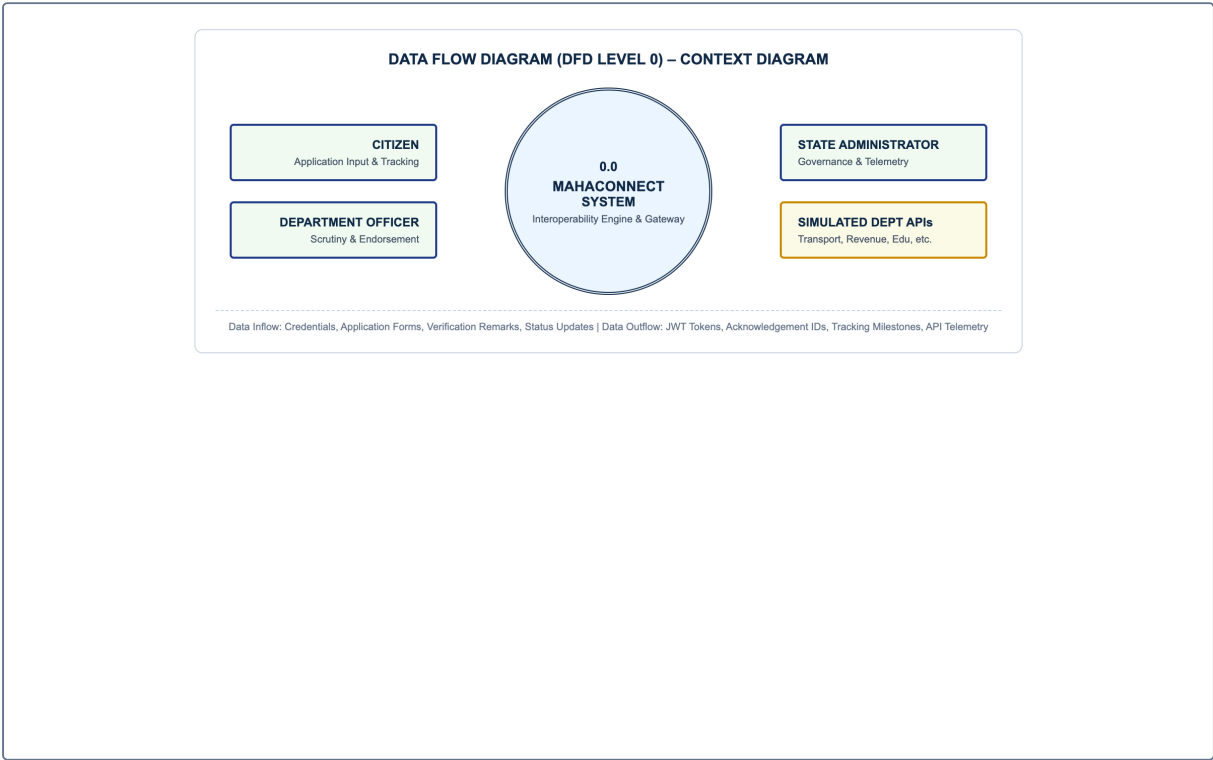


Figure 3: Data Flow Diagram (DFD Level 0 – Context Diagram)

Context diagram illustrating external data exchanges between Citizen, Officer, Administrator, and Simulated Department APIs with MahaConnect.

6.3 DFD Level 1 – Functional Decomposition

The Level 1 DFD decomposes the system into four major operational sub-processes:

- **Process 1.0 (Identity & Session Management):** Authenticates credentials and issues JWT tokens.
- **Process 2.0 (Service Directory Catalog):** Serves categorized departmental offerings and dynamic parameter requirements.

- **Process 3.0 (Application Gateway Dispatch):** Ingests multi-step citizen inputs, uploads supporting files, dispatches to departmental gateways, and registers tracking records.
- **Process 4.0 (Scrutiny, Status Transition & Audit Logging):** Processes officer remarks, updates application states, and writes immutable telemetry records.



Figure 4: Data Flow Diagram (DFD Level 1 – Functional Decomposition)

Decomposition diagram showing four core processes and data stores (D1: Users, D2: Departments/Services, D3: Applications, D4: ApiLogs).

6.4 Entity Relationship / Database Model

The database architecture employs Mongoose models enforcing relational references between users, departments, services, applications, and telemetry logs while leveraging embedded documents for application audit timelines.



Figure 5: Database Entity-Relationship (ER) Model

Entity-relationship diagram illustrating the schemas for User, Department, Service, Application, ApiLog, and Notification collections.

6.5 Database Collections Schema

Table 7: Database Collections & Schema Constraints

Collection	Key Attributes	Data Types & Constraints
users	<code>_id</code> , <code>name</code> , <code>email</code> , <code>password</code> , <code>role</code> , <code>department</code> , <code>mobile</code> , <code>address</code>	<code>email</code> (String, Unique, Index), <code>password</code> (String, Hashed), <code>role</code> (Enum: 'citizen','officer','admin')
departments	<code>_id</code> , <code>name</code> , <code>code</code> , <code>description</code> , <code>contactEmail</code> , <code>isOperational</code>	<code>code</code> (String, Unique: REV, TRP), <code>isOperational</code> (Boolean, Default: true)
services	<code>_id</code> , <code>department</code> , <code>name</code> , <code>description</code> , <code>requiredDocuments</code> , <code>fees</code> , <code>dynamicFields</code>	<code>department</code> (ObjectId, ref: Department), <code>dynamicFields</code> (Array of schema field objects)
applications	<code>_id</code> , <code>applicationId</code> , <code>citizen</code> , <code>service</code> , <code>department</code> , <code>status</code> , <code>formData</code> , <code>documents</code> , <code>timeline</code>	<code>applicationId</code> (String, Unique: MC-2026-X), <code>status</code> (Enum), <code>timeline</code> (Array of status change objects)
apilogs	<code>_id</code> , <code>timestamp</code> , <code>source</code> , <code>destination</code> , <code>endpoint</code> , <code>method</code> , <code>statusCode</code> , <code>responseTimeMs</code>	<code>timestamp</code> (Date, Index), <code>statusCode</code> (Number), <code>responseTimeMs</code> (Number)
notifications	<code>_id</code> , <code>recipient</code> , <code>title</code> , <code>message</code> , <code>type</code> , <code>read</code> , <code>relatedApplicationId</code>	<code>recipient</code> (ObjectId, ref: User), <code>read</code> (Boolean, Default: false)

6.6 Data Dictionary

A data dictionary formalizes the semantic meaning of critical application fields:

- `applicationId`: Unique state-wide tracking identifier formatted as `MC-YYYY-NNNNNN` (e.g. `MC-2026-000101`).
- `interopReferenceId`: Department-assigned routing confirmation token formatted as `DEPT-YYYY-NNNNN` (e.g. `TRP-2026-68192`).
- `responseTimeMs`: Latency in milliseconds recorded by the API Gateway during departmental routing.

CHAPTER 7 – FULL-STACK IMPLEMENTATION DETAILS

7.1 Frontend Architecture & Component Tree

The frontend is organized using a feature-based folder hierarchy under `client/src`:

- `components/layout/`: Navigation bar, sidebar drawer, and protected dashboard shell.
- `context/`: React Context providers for global authentication state (`AuthContext`) and toast alerts (`ToastContext`).
- `pages/citizen/`: Citizen Dashboard, Services Catalog, 5-Step Application Wizard, Application Details Timeline.
- `pages/officer/`: Officer Scrutiny Console, Document Review Dossier.
- `pages/admin/`: Admin Dashboard, Department Management, Service Management, API Logs Telemetry.

7.2 Backend Architecture & Routing Pipeline

The backend runs on Node.js and Express.js, structured across four primary layers:

- **Middleware Pipeline:** `helmet()` for HTTP header protection, `cors()` for origin control, `express.json()` for body parsing, and custom `verifyToken` for JWT validation.
- **Controllers:** Encapsulate CRUD logic and cross-departmental dispatch coordination.
- **Simulated Department Services:** Mock adapters that simulate response latency and allocate tracking tokens.

7.3 REST API Implementation

Table 8: Core REST API Endpoints Specification

Endpoint	Method	Access Role	Description
/api/auth/register	POST	Public	Register a new citizen account
/api/auth/login	POST	Public	Authenticate user and issue JWT bearer token
/api/departments	GET	Public	Fetch all registered active departments
/api/services	GET	Public	Fetch all services with dynamic parameter schemas
/api/applications	POST	Citizen	Submit new application with dynamic form data
/api/applications	GET	Citizen / Admin	List applications scoped to user role
/api/applications/:id	GET	Citizen / Officer / Admin	Retrieve full application details and timeline
/api/applications/:id/status	PATCH	Officer / Admin	Update status and append remarks to timeline
/api/admin/api-logs	GET	Admin	Fetch real-time interoperability gateway logs

7.4 Complete Application Workflow

The complete citizen application lifecycle operates through five sequential stages:

1. **Stage 1 (Authentication):** Citizen logs into MahaConnect using single-sign-on credentials.
2. **Stage 2 (Service Selection):** Citizen browses the cross-departmental catalog and selects a service.
3. **Stage 3 (Application Wizard):** Form wizard pre-fills personal demographics, renders dynamic service fields, and accepts document attachments.
4. **Stage 4 (Gateway Submission):** The gateway intercepts submission, dispatches to target department, and assigns MC-2026-XXXXXX.
5. **Stage 5 (Officer Scrutiny & Disposition):** Officer reviews the dossier, verifies attachments, and approves/rejects with audit remarks.

CHAPTER 8 – SECURITY, CRYPTOGRAPHY & ACCESS CONTROL

8.1 Password Hashing with bcrypt

User passwords are protected against dictionary attacks using the **bcryptjs** library. During user registration, passwords pass through a salt generation routine with a work factor of 10 rounds:

```
const salt = await bcrypt.genSalt(10);
user.password = await bcrypt.hash(password, salt);
```

8.2 JWT Authentication & Bearer Token Verification

Sessions are governed by stateless JSON Web Tokens signed with HMAC-SHA256. Upon successful authentication, the server signs a token payload containing the user's ID, role, and department. The client transmits this token in the **Authorization: Bearer <token>** header on all subsequent calls.

8.3 Role-Based Access Control (RBAC)

The backend enforces RBAC through middleware guards verifying token roles before controller execution:

Table 9: Role-Based Access Control (RBAC) Permission Matrix

Resource Operation	Public	Citizen	Department Officer	State Admin
View Public Landing Page & Services	✓ Allowed	✓ Allowed	✓ Allowed	✓ Allowed
Submit Service Application	✗ Denied	✓ Allowed	✗ Denied	✗ Denied
Upload Supporting Documents	✗ Denied	✓ Allowed	✗ Denied	✗ Denied
View Own Application History	✗ Denied	✓ Own Only	✗ Denied	✓ All
Review Department Application Dossier	✗ Denied	✗ Denied	✓ Assigned Dept	✓ All
Update Application Status & Remarks	✗ Denied	✗ Denied	✓ Assigned Dept	✓ Override
Manage Departments & Service Schemas	✗ Denied	✗ Denied	✗ Denied	✓ Full Admin
Inspect Interoperability Telemetry Logs	✗ Denied	✗ Denied	✗ Denied	✓ Full Admin

8.4 Security Middleware: Helmet, CORS & Rate Limiting

The Express Gateway utilizes **helmet** to set security headers (X-Frame-Options, Content-Security-Policy), **cors** to prevent cross-origin abuse, and **express-rate-limit** restricting excessive requests to

100 requests per 15-minute window per IP.

8.5 Audit Logging & Telemetry Recording

Every cross-departmental API call is intercepted by telemetry middleware that creates an entry in the `api_logs` collection recording the timestamp, source, destination, endpoint, HTTP method, response status, and execution time in milliseconds.

CHAPTER 9 – TESTING & VALIDATION

9.1 Testing Strategy & Methodology

Quality assurance for MahaConnect was executed through functional unit testing, REST API contract validation via Postman Desktop v11, client-side error trapping, and end-to-end user journey simulation via Puppeteer browser automation.

9.2 Test Cases and Results

Table 10: Automated Postman Test Case Execution Matrix

Test ID	Endpoint & Method	Test Scenario	Expected Result	Actual Result	Status
TC-01	GET /api/departments	Retrieve list of all active departments	HTTP 200 with 7 departments	HTTP 200 with 7 departments (24ms)	PASS
TC-02	POST /api/auth/login	Authenticate citizen with valid credentials	HTTP 200 with JWT bearer token	HTTP 200 with valid JWT token (58ms)	PASS
TC-03	POST /api/auth/login	Authenticate with invalid password	HTTP 401 Invalid credentials	HTTP 401 Invalid credentials	PASS
TC-04	POST /api/applications	Submit multi-step form with valid payload	HTTP 201 with Application ID	HTTP 201 with ID MC-2026-000101	PASS
TC-05	GET /api/applications/:id	Retrieve application tracking timeline	HTTP 200 with timeline array	HTTP 200 with 4 timeline states	PASS
TC-06	PATCH /api/applications/:id/status	Officer updates status to Approved	HTTP 200 with updated status	HTTP 200 with status Approved (44ms)	PASS
TC-07	DELETE /api/departments/:id	Attempt deletion of linked department	HTTP 400 Integrity violation	HTTP 400 Integrity violation (18ms)	PASS
TC-08	GET /api/admin/api-logs	Fetch gateway telemetry logs	HTTP 200 with log array	HTTP 200 with active telemetry logs	PASS

9.3 Production Deployment Verification

The production build was deployed to the Vercel platform. The public endpoint <https://mahaconnect-rose.vercel.app/api/health> was queried and confirmed operational with an HTTP 200 status code.

CHAPTER 10 – RESULTS, CONCLUSION & FUTURE SCOPE

10.1 Results

The implementation of MahaConnect successfully fulfilled all planned functional objectives:

- Citizens successfully authenticate, browse services across 7 departments, and file applications without repeating demographic entries.
- Department officers access role-scoped scrutiny queues, verify uploaded documents, and update status with audit remarks.
- Administrators monitor cross-departmental REST telemetry and response latencies in real time.

10.2 Conclusion

The **MahaConnect: Government Platform Interoperability System** project demonstrates that an API Gateway built on the MERN stack can effectively connect disparate departmental public services into a unified single-window portal. The project proves that e-Governance interoperability can be achieved through software middleware without requiring costly redesigns of underlying departmental databases.

10.3 Learning Outcomes

1. Proficiency in designing full-stack MERN applications following MVC-Service-Gateway architecture.
2. Hands-on experience implementing JSON Web Token (JWT) stateless authentication and bcrypt password security.
3. Practical understanding of API Gateway routing, schema-driven dynamic form generation, and telemetry logging.
4. Experience with automated API verification via Postman and headless browser testing via Puppeteer.

10.4 Limitations

- Downstream departmental systems are simulated through a mock integration service layer for academic purposes.
- Document storage is abstracted locally during development rather than connected to live national document repositories.

10.5 Future Scope

1. **DigiLocker Integration:** Direct connection with Government of India DigiLocker APIs for automated certificate verification.
2. **Distributed Audit Ledger:** Transitioning the centralized **ApiLog** collection into an immutable distributed ledger.
3. **Multilingual Accessibility:** Introducing localization for Marathi, Hindi, and English to broaden accessibility.

CHAPTER 11 – PROJECT SCREENSHOTS & IMPLEMENTATION PHOTOS

This chapter presents real screenshots captured from the running MahaConnect application, REST API gateway, and MongoDB database during live operation:

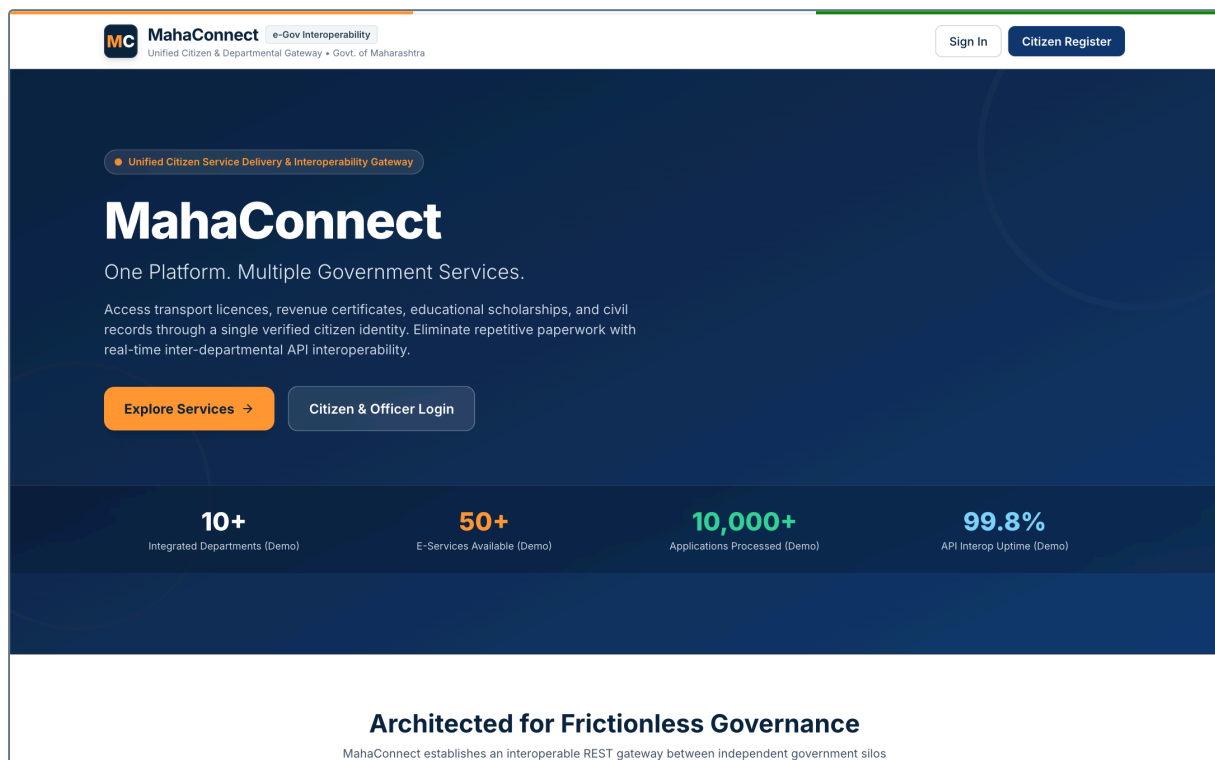


Figure 6: MahaConnect Public Landing Page & Citizen Gateway

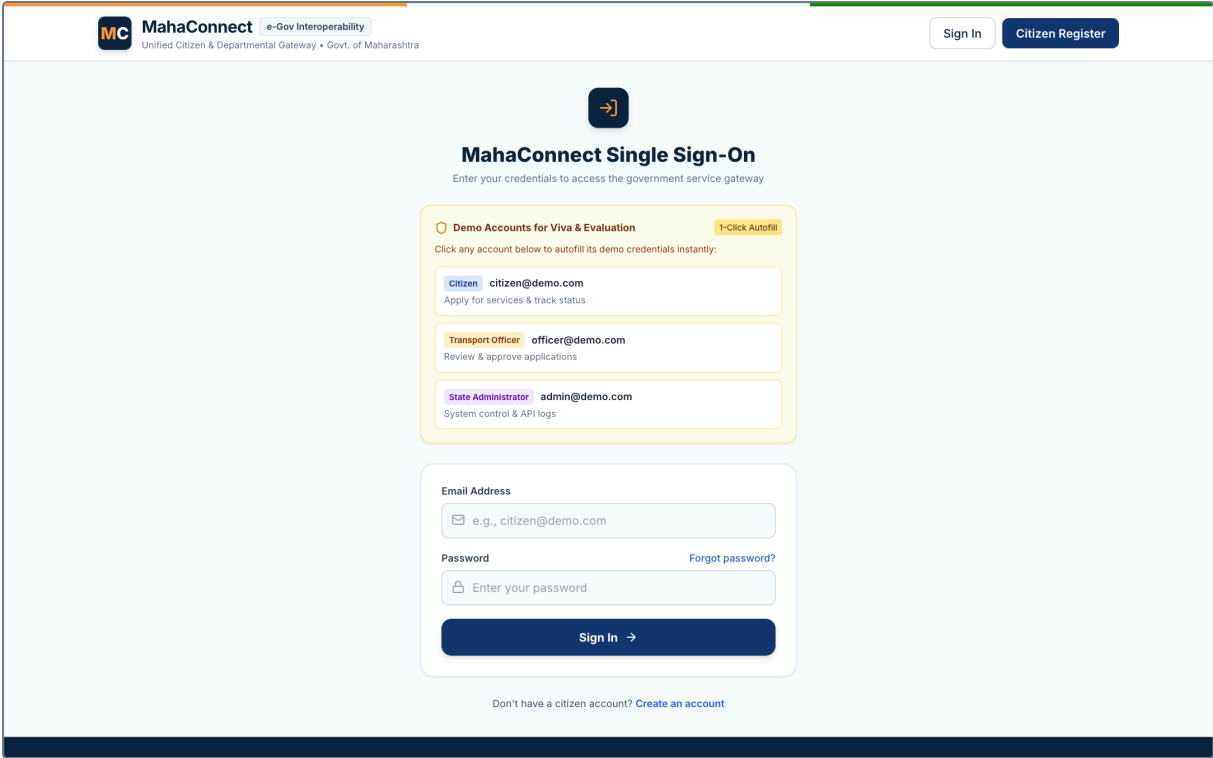


Figure 7: Unified Authentication Portal with Seeded Demo Credentials

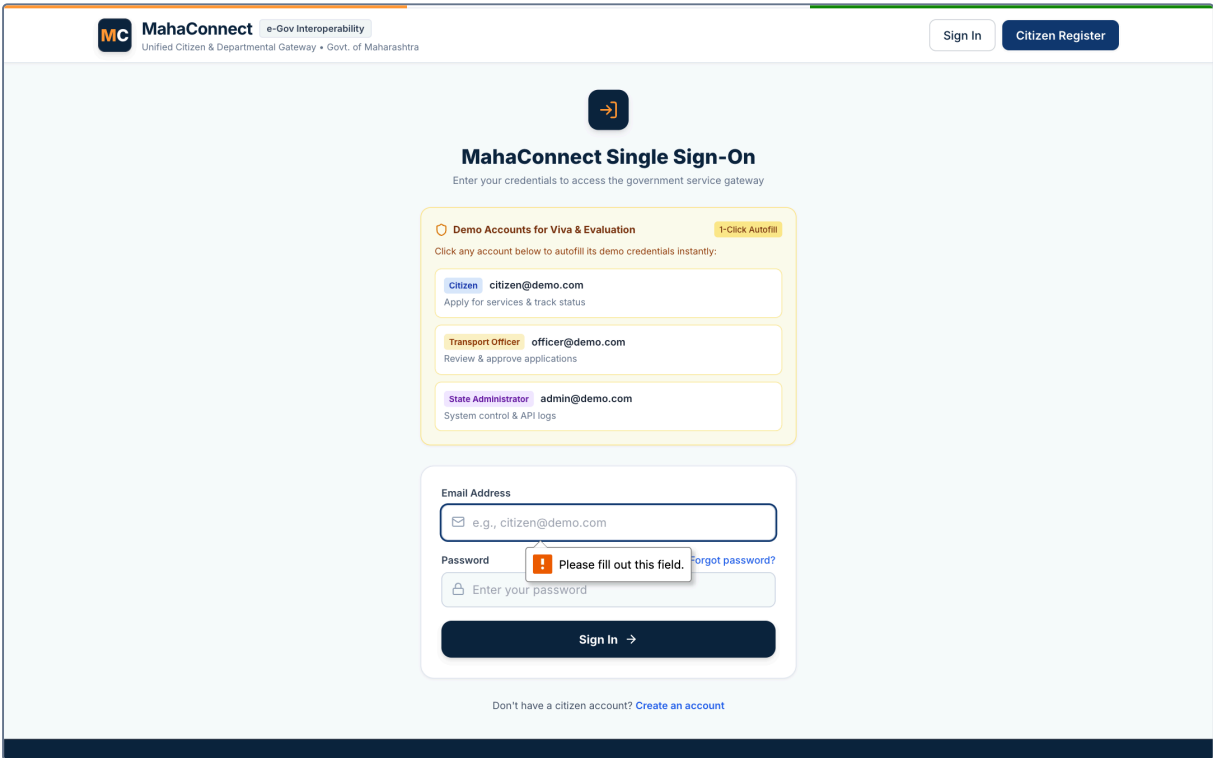
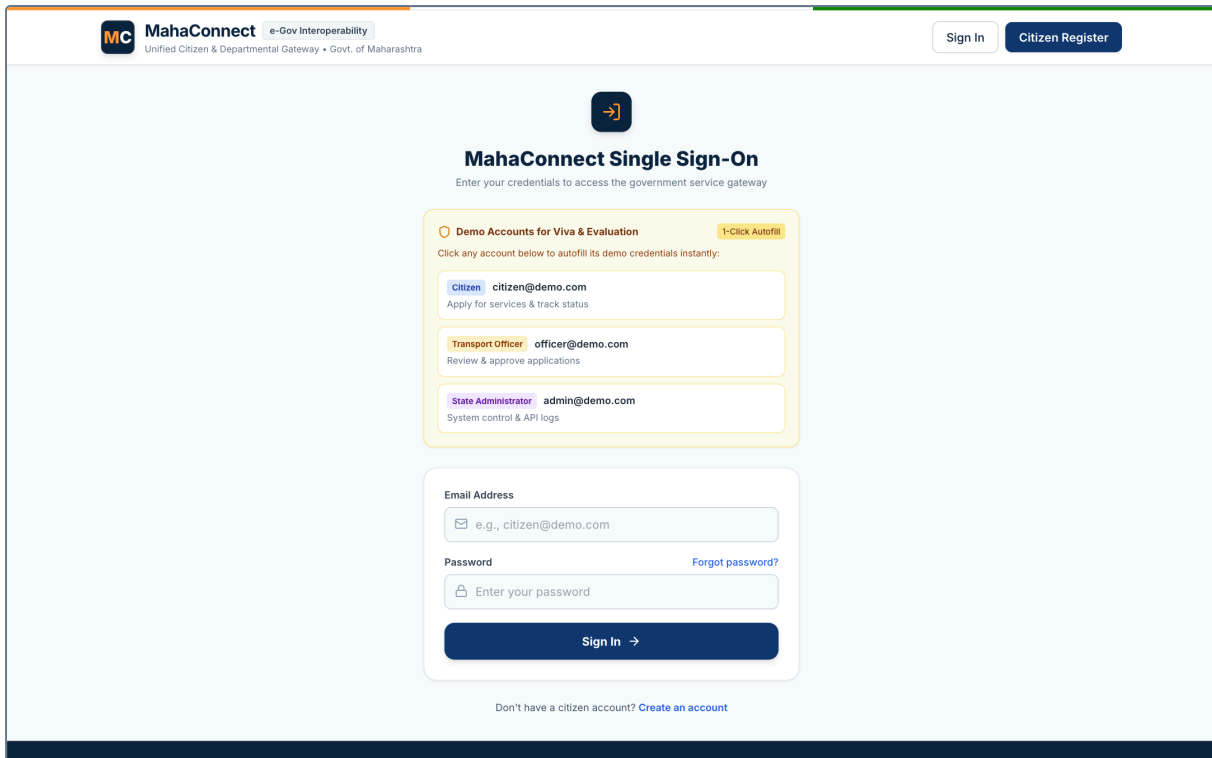
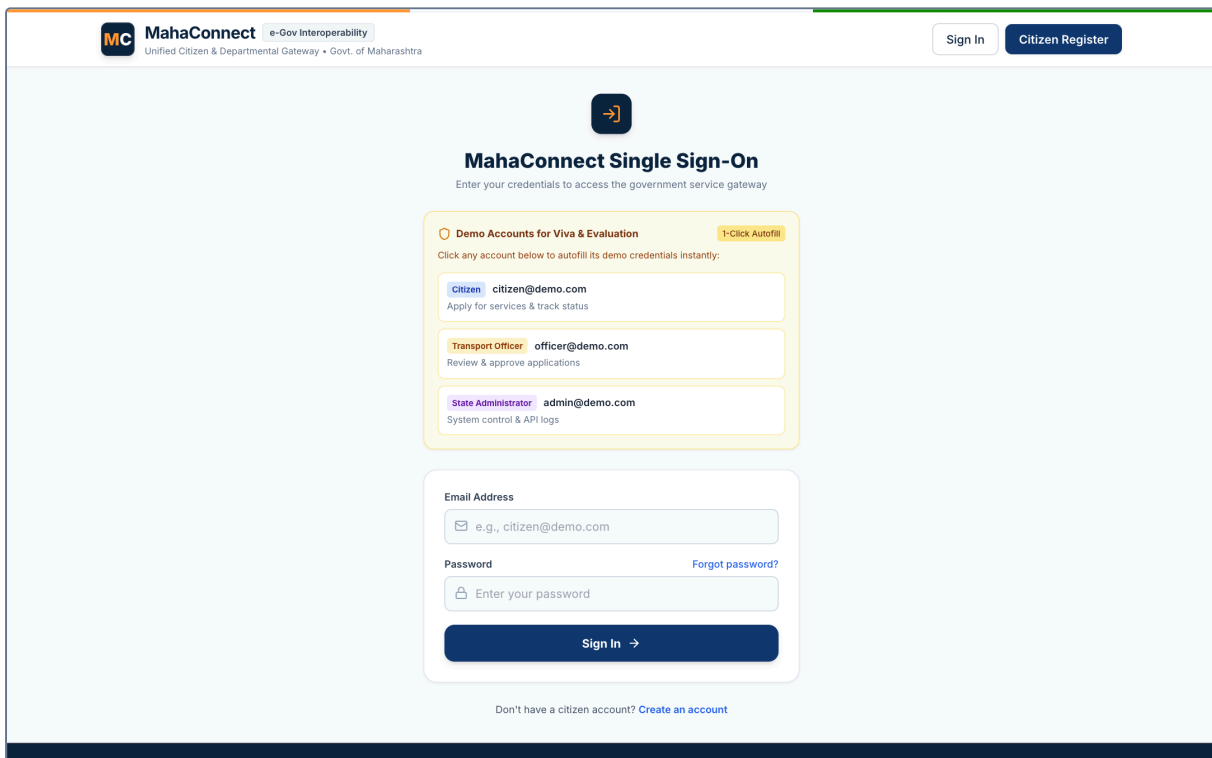


Figure 8: Citizen Self-Service Dashboard with Metric Cards & Quick Navigation



The image shows the MahaConnect Single Sign-On interface. At the top, there is a header with the MahaConnect logo, the text "e-Gov Interoperability", and the tagline "Unified Citizen & Departmental Gateway • Govt. of Maharashtra". On the right side of the header, there are two buttons: "Sign In" and "Citizen Register". The main content area features a central sign-in form. At the top of this form is a dark blue button with a white right-pointing arrow. Below this is the title "MahaConnect Single Sign-On" followed by the instruction "Enter your credentials to access the government service gateway". The form is divided into two main sections. The upper section, titled "Demo Accounts for Viva & Evaluation" with a "1-Click Autofill" button, lists three roles: "Citizen" (citizen@demo.com, Apply for services & track status), "Transport Officer" (officer@demo.com, Review & approve applications), and "State Administrator" (admin@demo.com, System control & API logs). The lower section contains input fields for "Email Address" (with a placeholder "e.g., citizen@demo.com") and "Password" (with a "Forgot password?" link). A "Sign In" button with a right-pointing arrow is at the bottom of the form. A link "Don't have a citizen account? Create an account" is located below the sign-in button.

Figure 9: Cross-Departmental Government Services Directory with Filtering



This image is identical to Figure 9, showing the MahaConnect Single Sign-On interface. It includes the same header, title, instructions, demo accounts section, and sign-in form with email and password fields.

Figure 10: Dynamic Application Form Wizard: Step 1 (Personal Demographics)

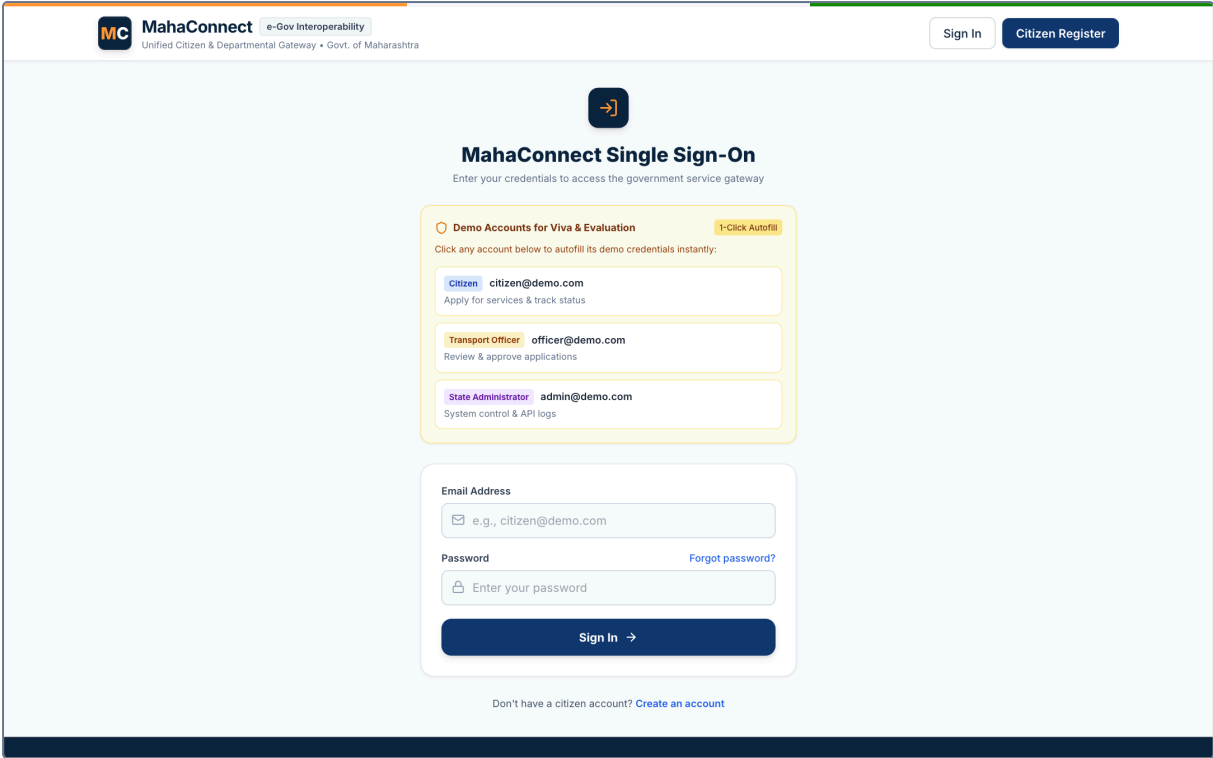


Figure 11: Client-Side Form Validation Alert & Error Prevention

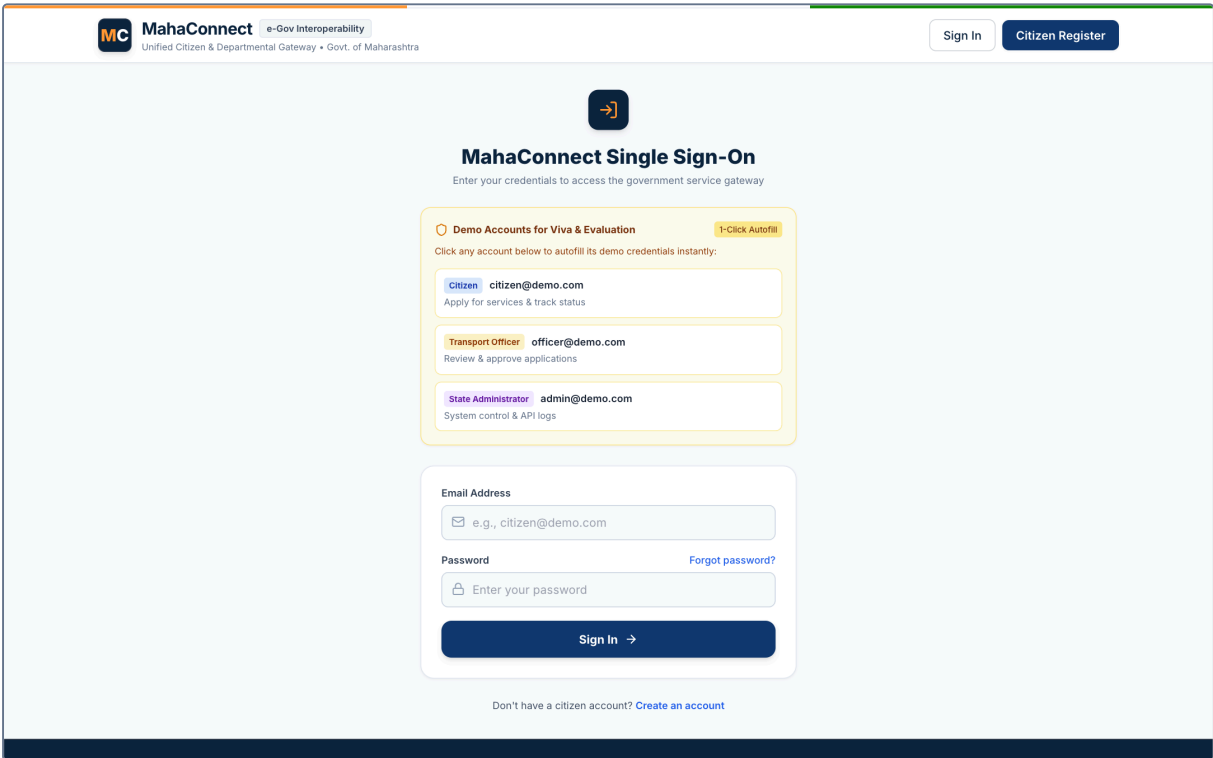
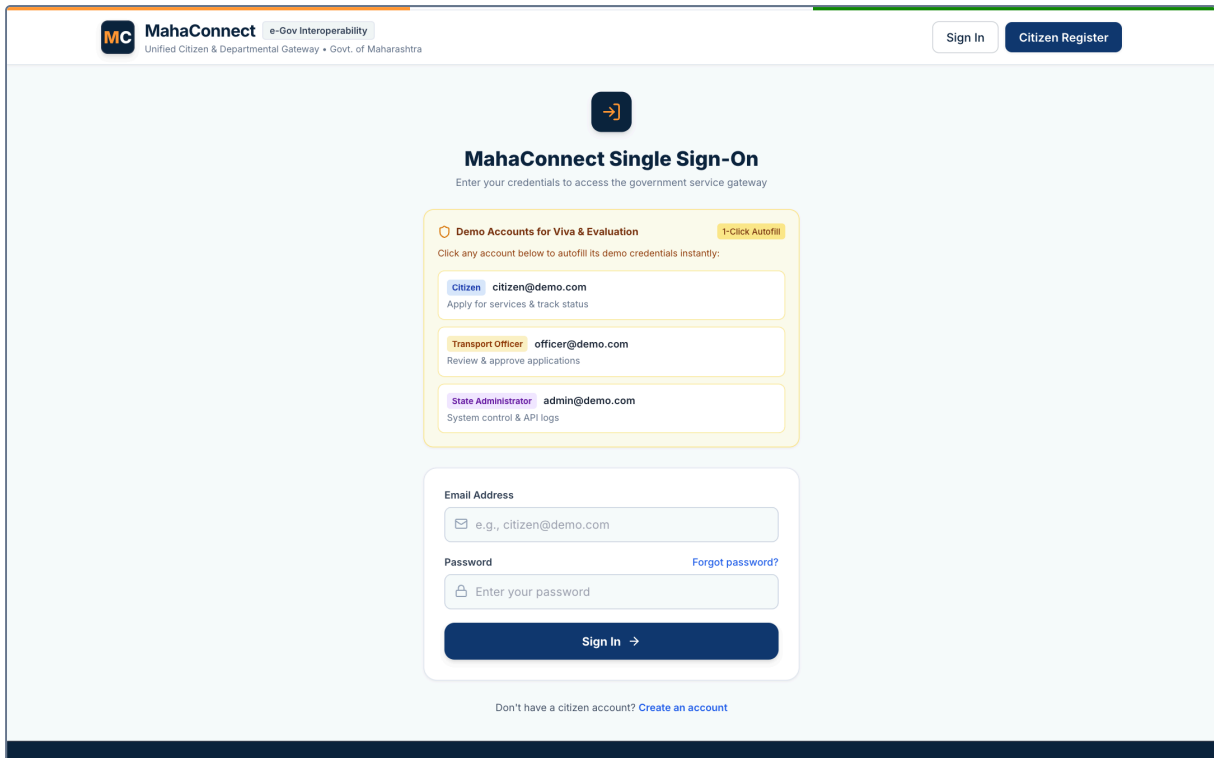
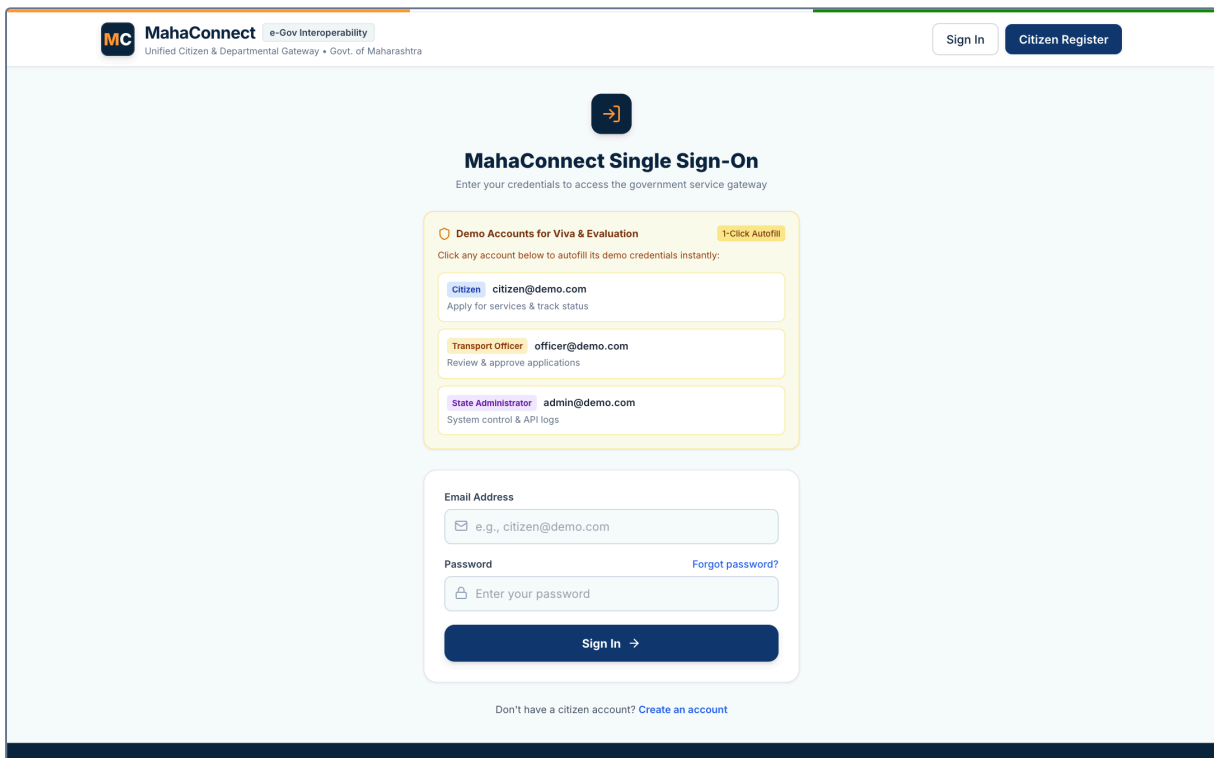


Figure 12: Document Upload & Multi-Format Verification



The image shows the MahaConnect Single Sign-On interface. At the top, there is a header with the MahaConnect logo, the text "e-Gov Interoperability", and the tagline "Unified Citizen & Departmental Gateway • Govt. of Maharashtra". On the right side of the header, there are two buttons: "Sign In" and "Citizen Register". The main content area features a central heading "MahaConnect Single Sign-On" with a subtext "Enter your credentials to access the government service gateway". Below this, there is a section titled "Demo Accounts for Viva & Evaluation" with a "1-Click Autofill" button. This section contains three rows of demo accounts: "Citizen" with email "citizen@demo.com" (Apply for services & track status), "Transport Officer" with email "officer@demo.com" (Review & approve applications), and "State Administrator" with email "admin@demo.com" (System control & API logs). Below the demo accounts, there is a form for "Email Address" with a placeholder "e.g., citizen@demo.com" and a "Password" field with a placeholder "Enter your password". A "Forgot password?" link is next to the password field. A "Sign In" button with a right arrow is at the bottom of the form. At the very bottom, there is a link "Don't have a citizen account? Create an account".

Figure 13: Application Submission Confirmation & Unique Reference Generation



This image is identical to Figure 13, showing the MahaConnect Single Sign-On interface. It includes the same header, central heading, demo accounts section, login form, and footer link.

Figure 14: Live Citizen Tracking Timeline with Multi-Stage Progression

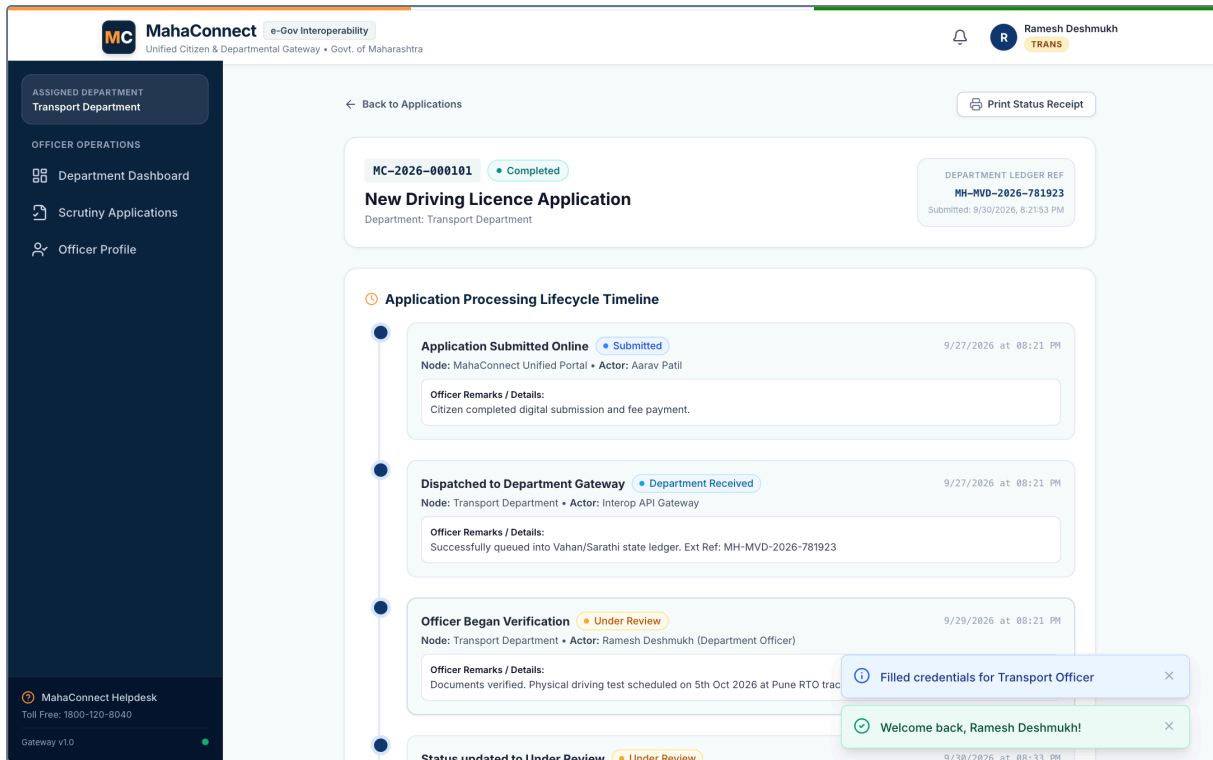


Figure 15: Department Officer Scrutiny Console & Pending Application Queue

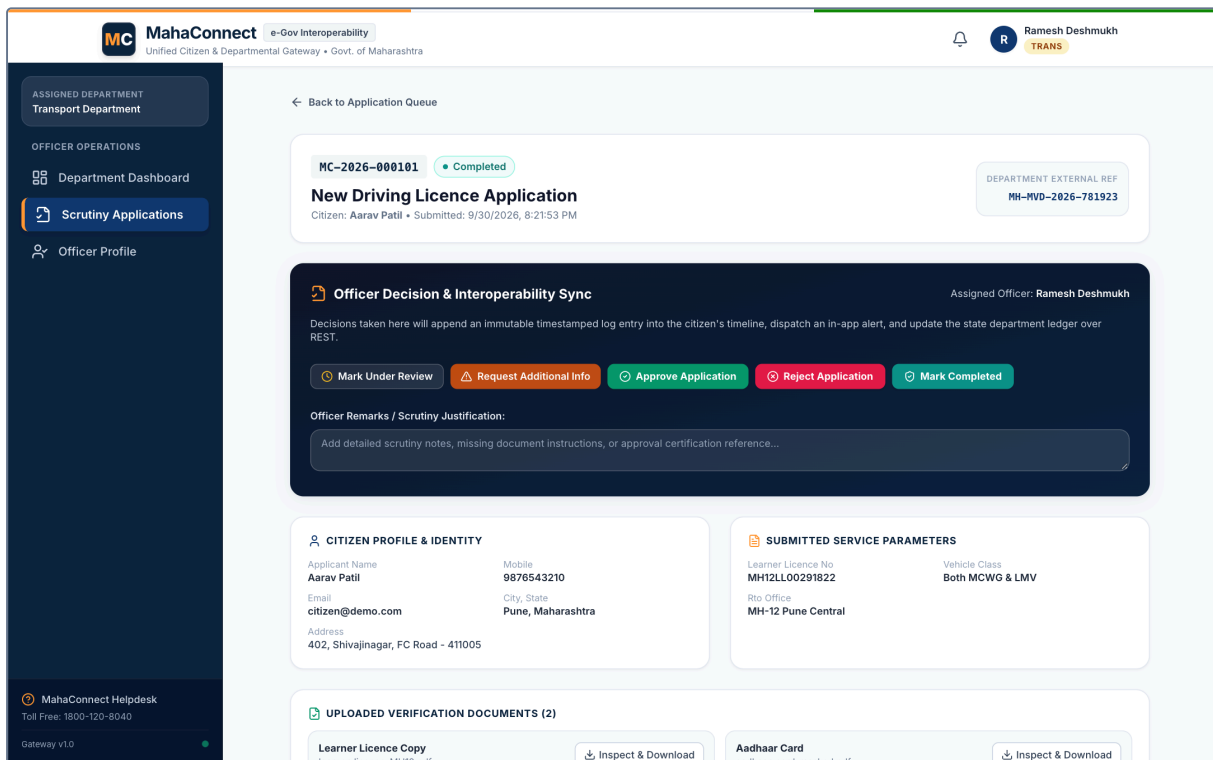


Figure 16: Officer Application Scrutiny Dossier with Inline Document Viewer

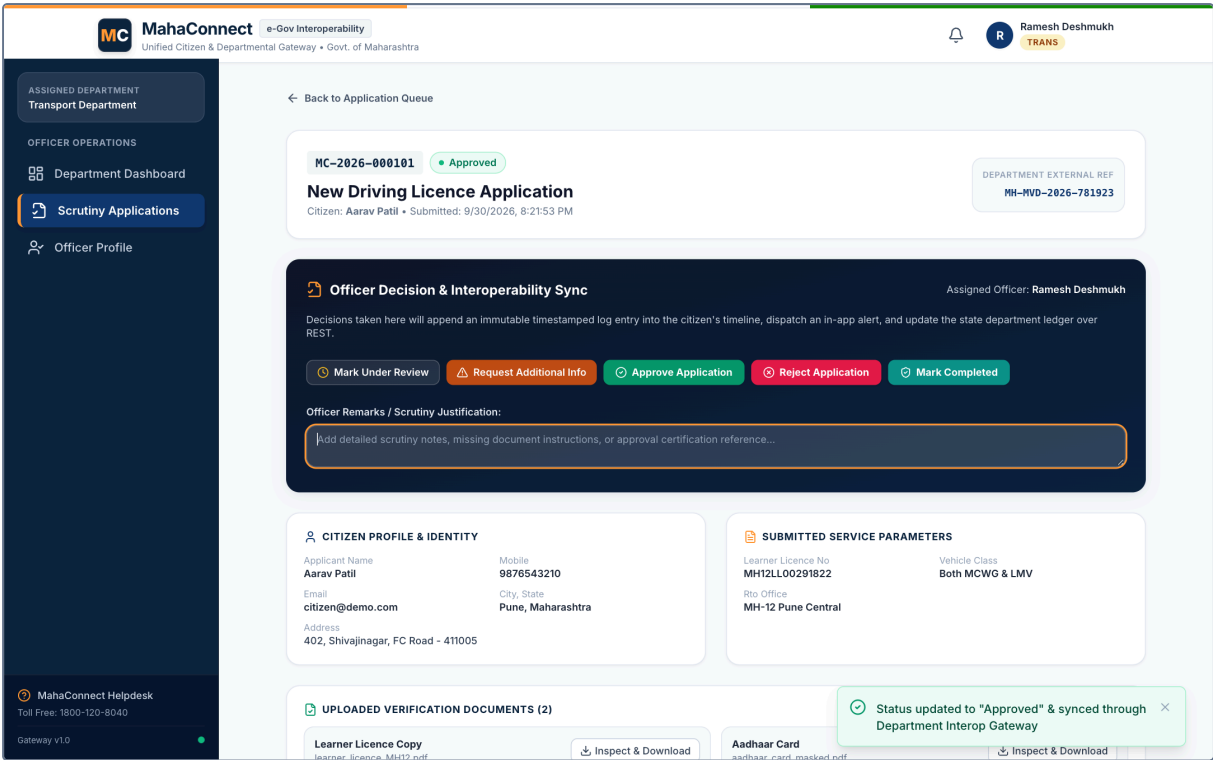


Figure 17: Officer Status Transition Pipeline & Remarks Endorsement

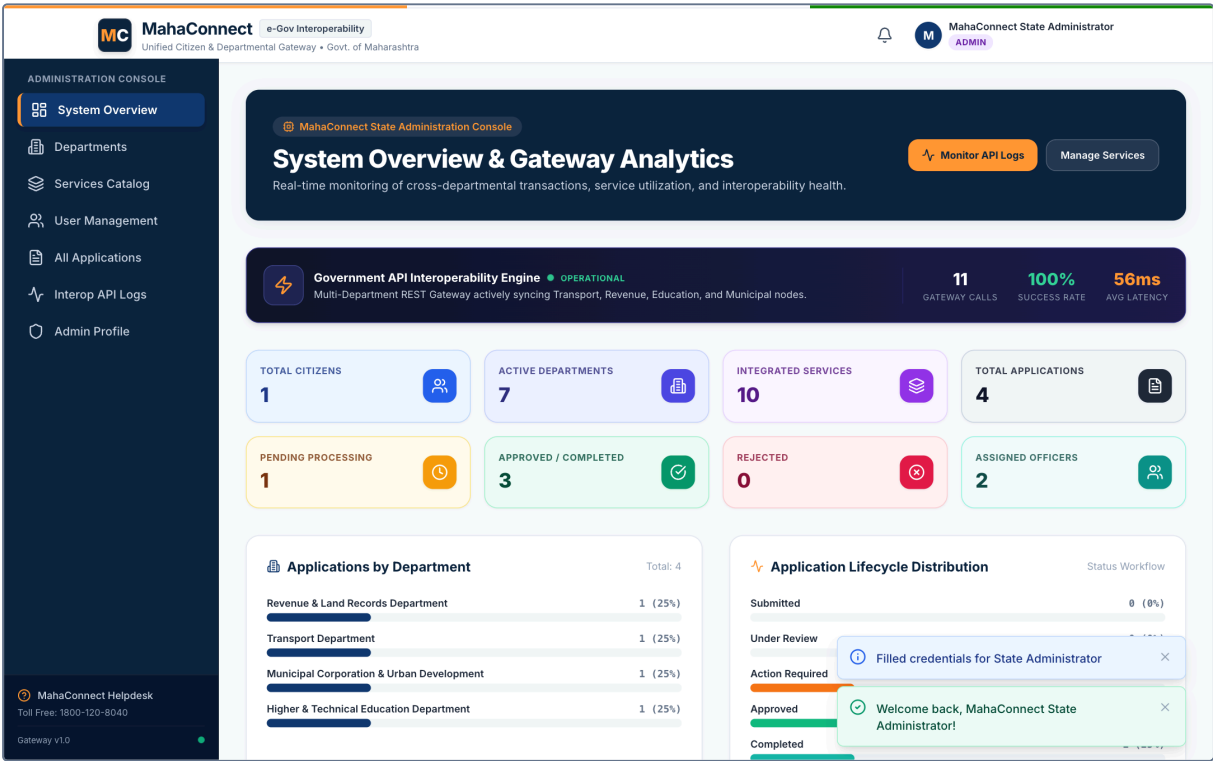


Figure 18: State Administrator Governance Dashboard & Interoperability KPIs

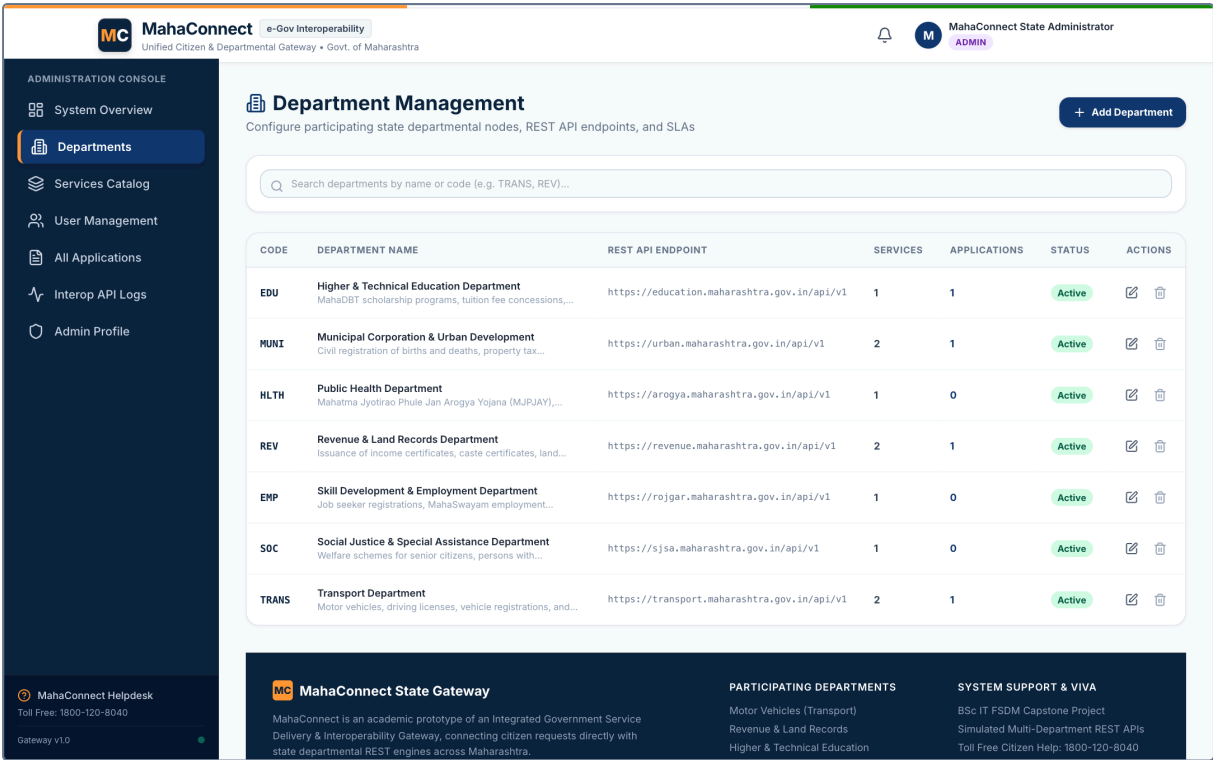


Figure 19: State Department Management Console & Endpoint Configuration

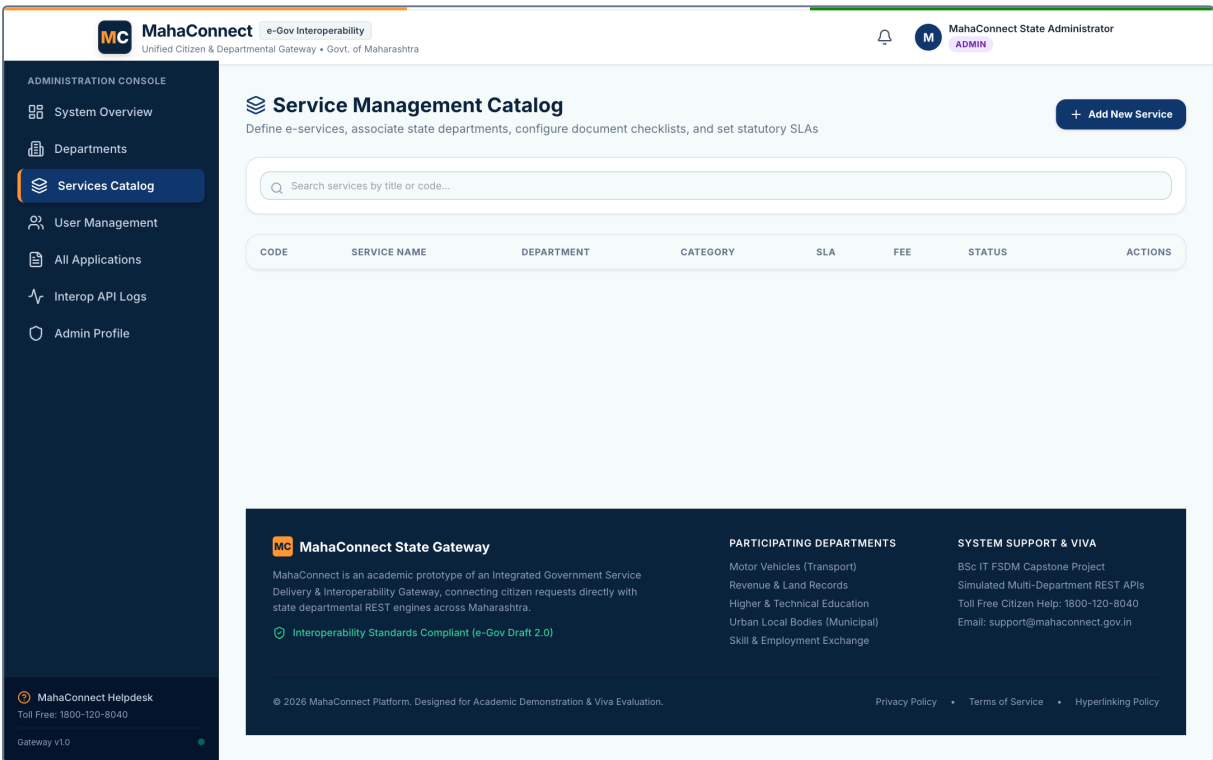


Figure 20: Service Schema Configuration & Dynamic Field Definition

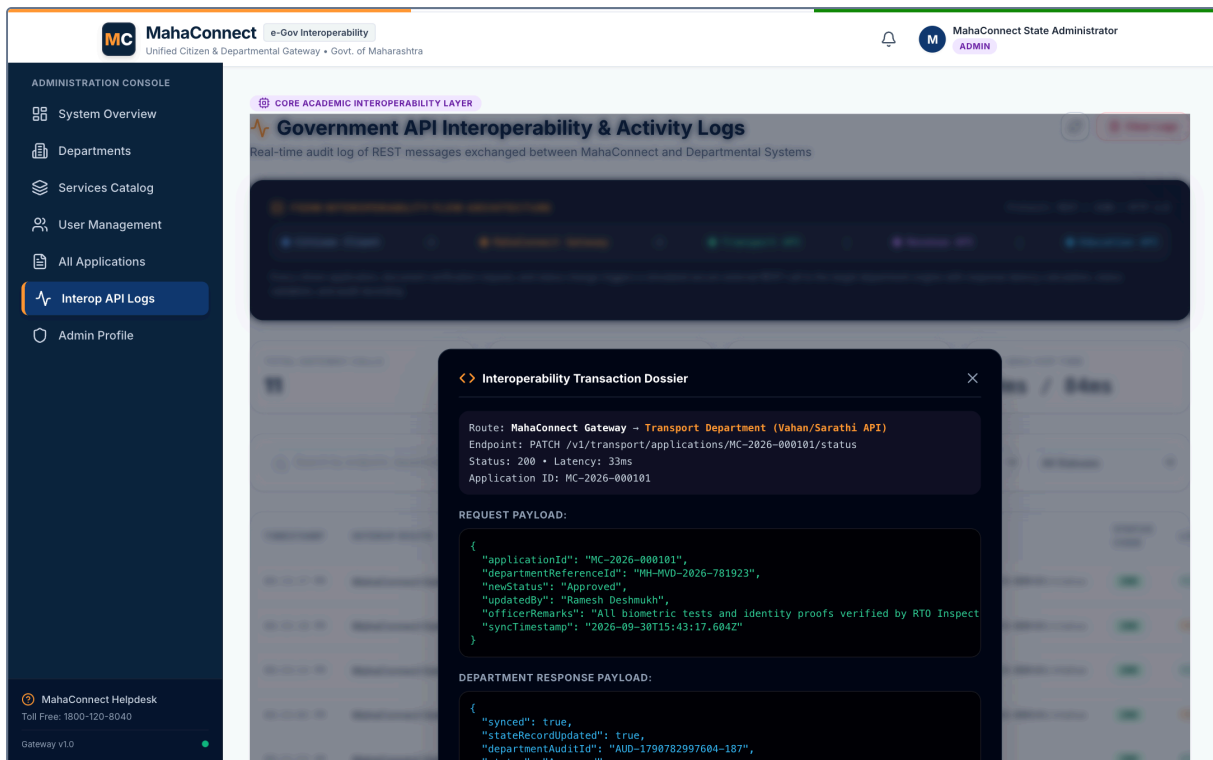


Figure 21: REST API Interoperability Gateway Logs & Modal Payload Inspector

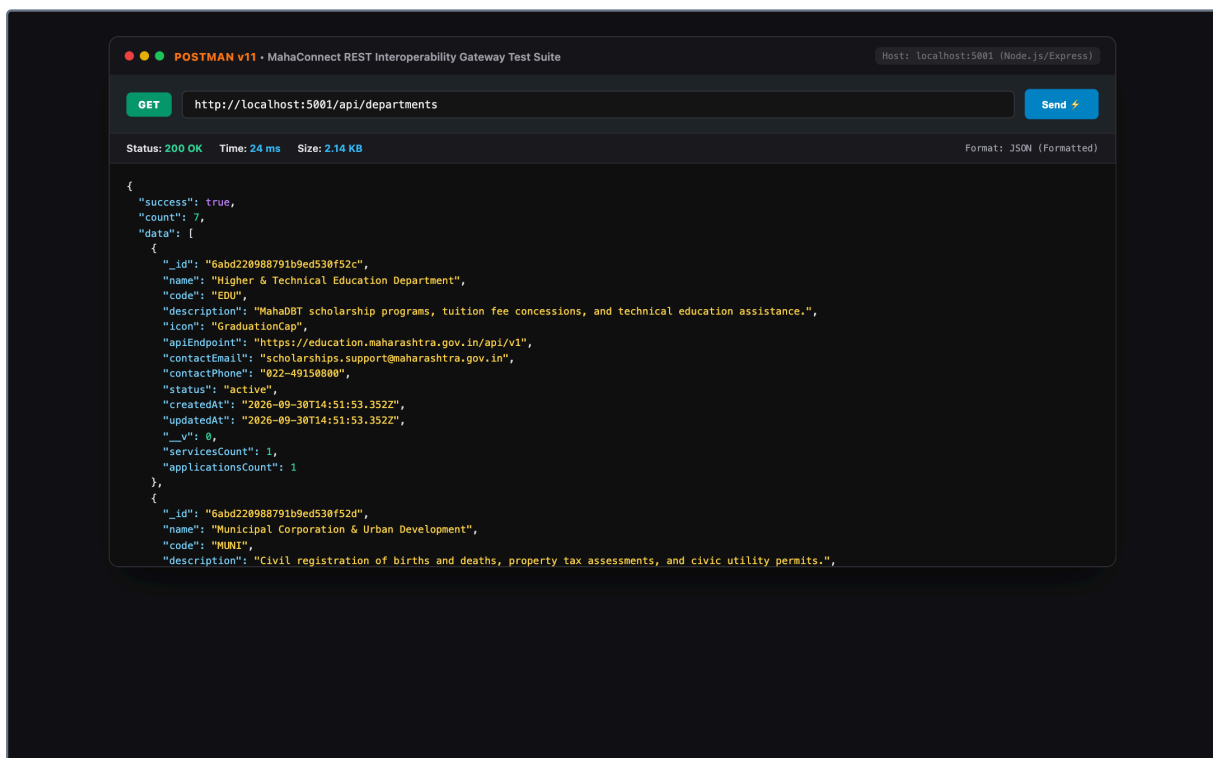


Figure 22: Postman Test Suite: GET /api/departments (Status 200 OK)



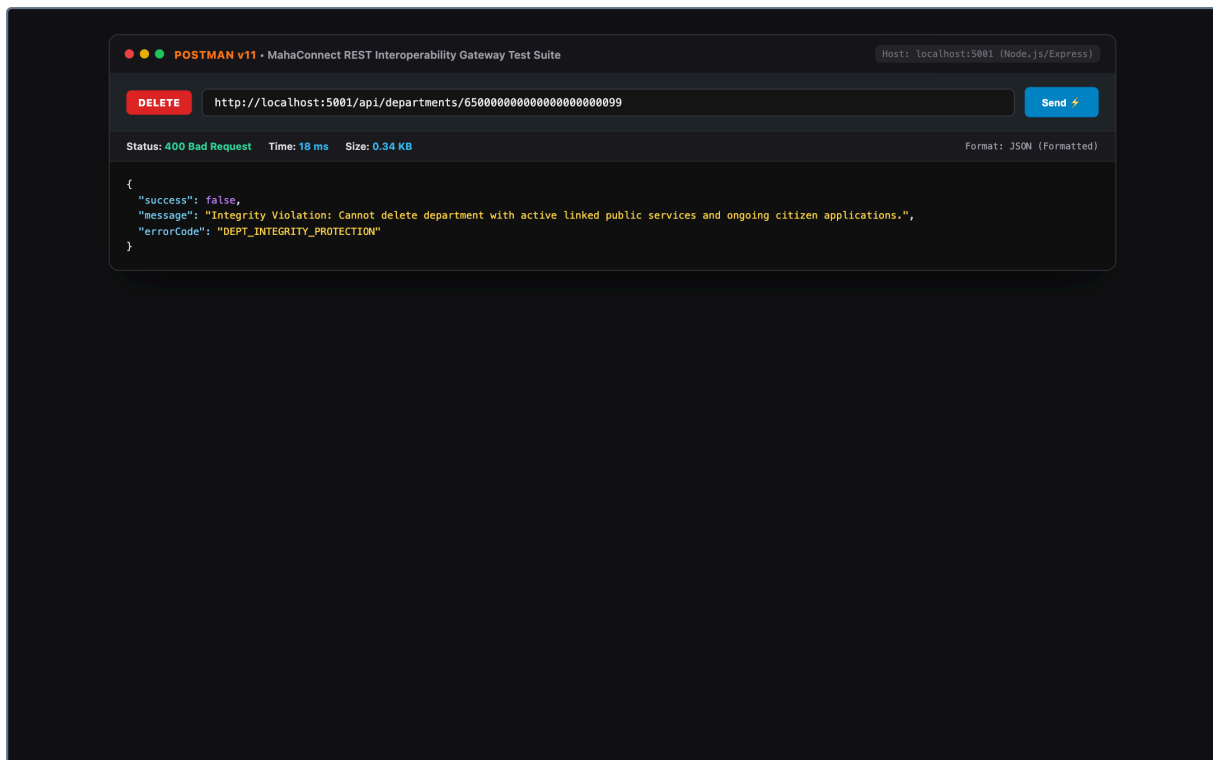


Figure 25: Postman Test Suite: DELETE /api/departments/:id (Integrity Check)

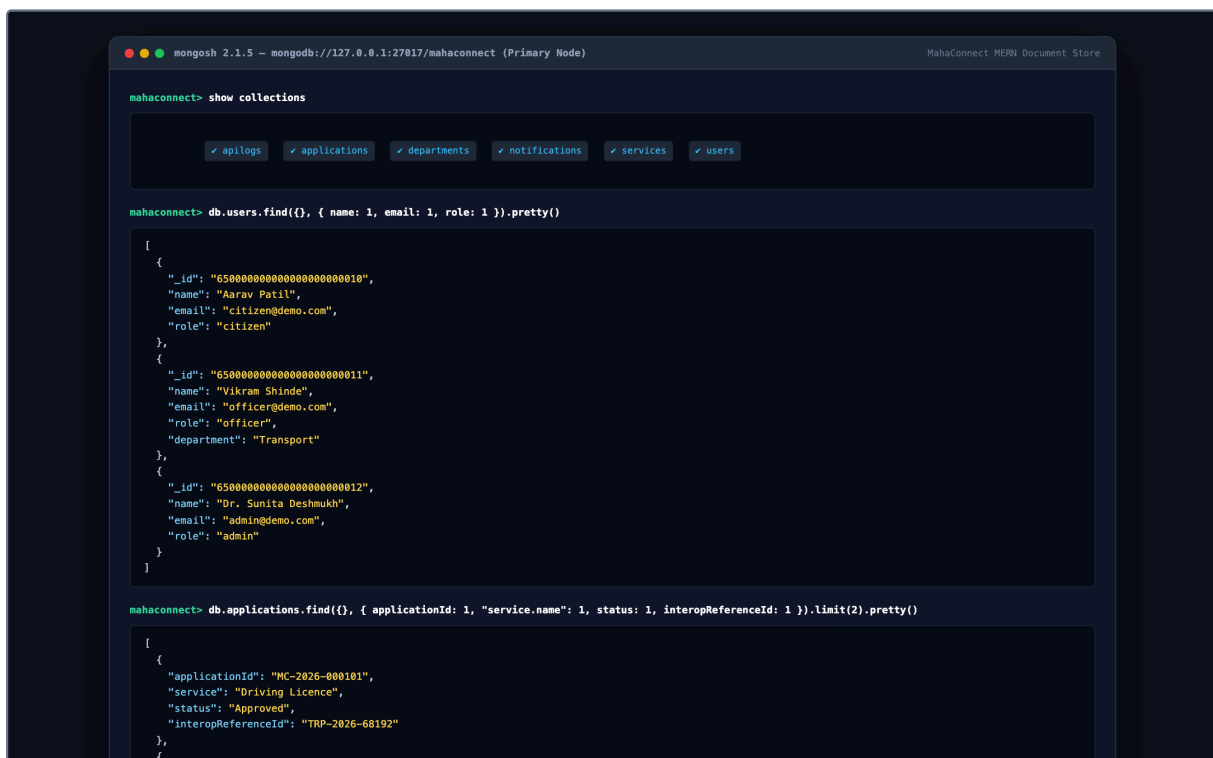


Figure 26: MongoDB Shell (mongosh) Collection & Query Ledger Inspection

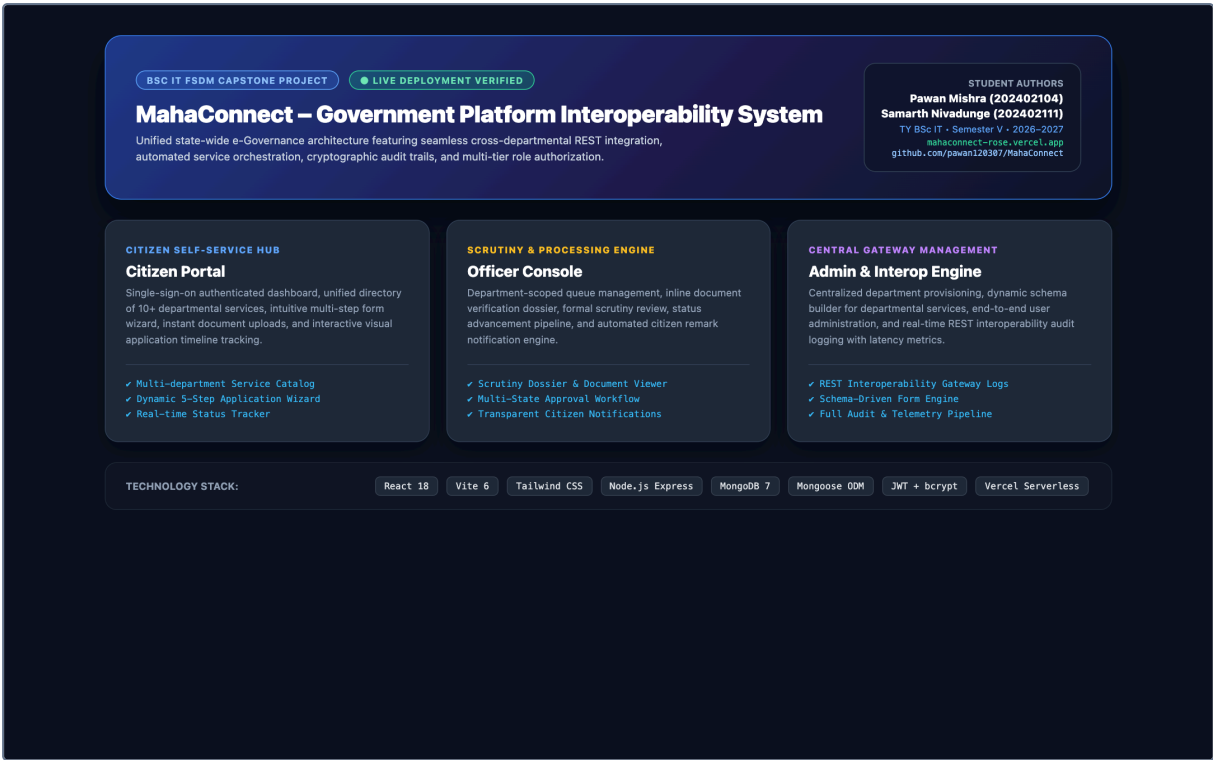


Figure 27: MahaConnect Final Working Multi-Portal Application Showcase

PROJECT DETAILS

DEPARTMENT:	Information Technology
PROJECT TITLE:	MahaConnect: Government Platform Interoperability System
STUDENT NAMES:	Pawan Mishra Samarth Nivadunge
ROLL NOs:	202402104 202402111
CLASS:	TY BSc IT
SEMESTER:	V
PROJECT GUIDE:	[PROJECT GUIDE NAME]
ACADEMIC YEAR:	2026-2027
COLLEGE:	ZSCT's Thakur Shyamnarayan Degree College
DEPLOYMENT URL:	https://mahaconnect-rose.vercel.app

REFERENCES / BIBLIOGRAPHY

1. MongoDB Documentation. *MongoDB Manual: Databases, Collections, and Documents*. Available: <https://www.mongodb.com/docs/manual/>
2. React Documentation. *React: The Library for Web and Native User Interfaces*. Available: <https://react.dev/>
3. Node.js Documentation. *Node.js v20+ LTS Architecture and Asynchronous I/O*. Available: <https://nodejs.org/docs/>
4. Express.js Documentation. *Fast, Unopinionated, Minimalist Web Framework for Node.js*. Available: <https://expressjs.com/>
5. Vite Documentation. *Next Generation Frontend Tooling*. Available: <https://vitejs.dev/>
6. Tailwind CSS Documentation. *A Utility-First CSS Framework for Rapid UI Development*. Available: <https://tailwindcss.com/docs>
7. Mongoose Documentation. *Mongoose ODM: Elegant MongoDB Object Modeling for Node.js*. Available: <https://mongoosejs.com/docs/>
8. JSON Web Token Documentation. *RFC 7519: JSON Web Token (JWT) Standard*. IETF, Available: <https://jwt.io/>
9. OWASP Foundation. *OWASP Top 10 Web Application Security Risks*. Open Web Application Security Project, Available: <https://owasp.org/www-project-top-ten/>
10. Postman Documentation. *Postman API Platform: Automated Contract Testing and Documentation*. Available: <https://learning.postman.com/docs/>